Act2_actualizado

December 9, 2025

1 CLASIFICACION CON SVM Y NN

En la presente actividad se va a emplear un dataset que contiene información de características de celulares para clasificarlos en un rango de precios.

La información del *dataset* es: * *battery_power*: Energía total que una batería puede almacenar en un tiempo medido en *mAh* * *blue*: Indica si tiene *bluetooth* o no * *clock_speed*: Velocidad del microprocesador * *dual_sim*: Indica si tiene soporte dual de *sim* o no * *fc*: Megapíxeles de la cámara frontal * *four_g*: Indica si tiene 4G o no * *int_memory*: Memoria interna en GB * *m_dep*: Grosor del celular en cm * *mobile_wt*: Peso del celular * *n_cores*: Número de núcleos del procesador * *pc*: Megapíxeles de la cámara principal * *px_height*: Alto de la resolución de píxeles * *px_width*: Ancho de resolución de píxeles * *ram*: RAM en MB * *sc_h*: Alto de la pantalla en cm * *sc_w*: Ancho de la pantalla en cm * *talk_time*: tiempo máximo que durará una sola carga de la batería * *tres_g*: Indica si es 3G o no * *touch_screen*: Indica si tiene pantalla táctil o no * *wifi*: Indica si tiene wifi o no * ***price_range***: Es la variable objetivo con valor 0 (bajo costo), 1 (costo medio), 2 (costo alto) y 3 (costo muy alto).

---

2 Recomendaciones generales

1. **Establece una semilla aleatoria** para garantizar la reproducibilidad de los resultados (la semilla se definirá más adelante).
2. **Lee el dataset solo una vez**, al inicio del notebook. Evita cargarlo repetidamente.
3. **Usa exclusivamente los hiperparámetros indicados** en cada modelo. Por ejemplo, en el primer SVM utiliza C=1. No modifiques ningún valor a menos que se solicite explícitamente.
4. **Organiza tu notebook** incluyendo celdas para: código, visualizaciones y análisis textual (explicaciones, interpretaciones, etc.).
5. **Responde de manera clara y completa**, siguiendo las instrucciones de cada ejercicio. Cuida tu redacción, ortografía y coherencia.
6. **Ante dudas o ambigüedades**, consulta a través de los canales de comunicación establecidos para resolverlas oportunamente.
7. **Antes de entregar tu actividad**, reinicia el kernel y ejecuta todas las celdas nuevamente para verificar la coherencia de los resultados.

2.0.1 IMPORTACIÓN DE LIBRERIAS A UTILIZAR

[1]:
```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score, confusion_matrix,␣
↪classification_report
```

2.0.2 Definición de semilla

[2]:
```
# La semilla que se va a emplear en la presente actividad es 1234, la cual no␣
↪debes modificar, para garantizar que los resultados que reportes coincidan␣
↪con los esperados

seed = 1234
```

2.0.3 CARGA DEL DATASET

[ ]:
```
# from google.colab import drive
# drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

[3]:
```
data = pd.read_csv('../datasets/train.csv')
data
```

[3]:
```
      battery_power  blue  clock_speed  dual_sim  fc  four_g  int_memory  \
0               842     0          2.2         0   1       0           7
1              1021     1          0.5         1   0       1          53
2               563     1          0.5         1   2       1          41
3               615     1          2.5         0   0       0          10
4              1821     1          1.2         0  13       1          44
…               …    …           …         …  ..      …          …
1995            794     1          0.5         1   0       1           2
1996           1965     1          2.6         1   0       0          39
1997           1911     0          0.9         1   1       1          36
1998           1512     0          0.9         0   4       1          46
1999            510     1          2.0         1   5       1          45
```

```
      m_dep  mobile_wt  n_cores  …  px_height  px_width   ram  sc_h  sc_w  \
0       0.6        188        2  …         20       756  2549     9     7
1       0.7        136        3  …        905      1988  2631    17     3
2       0.9        145        5  …       1263      1716  2603    11     2
3       0.8        131        6  …       1216      1786  2769    16     8
4       0.6        141        2  …       1208      1212  1411     8     2
…     …         …       …  …         …        …   …    …    …
1995    0.8        106        6  …       1222      1890   668    13     4
1996    0.2        187        4  …        915      1965  2032    11    10
1997    0.7        108        8  …        868      1632  3057     9     1
1998    0.1        145        5  …        336       670   869    18    10
1999    0.9        168        6  …        483       754  3919    19     4

      talk_time  three_g  touch_screen  wifi  price_range
0            19        0             0     1            1
1             7        1             1     0            2
2             9        1             1     0            2
3            11        1             0     0            2
4            15        1             1     0            1
…          …       …            …   …          …
1995         19        1             1     0            0
1996         16        1             1     1            2
1997          5        1             1     0            3
1998         19        1             1     1            0
1999          2        1             1     1            3

[2000 rows x 21 columns]
```

[4]:
```
#Tamaño del dataset
data.shape
```

[4]:
```
(2000, 21)
```

[5]:
```
#Información de las variables
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   battery_power  2000 non-null   int64
 1   blue           2000 non-null   int64
 2   clock_speed    2000 non-null   float64
 3   dual_sim       2000 non-null   int64
 4   fc             2000 non-null   int64
```

```
 5   four_g         2000 non-null   int64
 6   int_memory     2000 non-null   int64
 7   m_dep          2000 non-null   float64
 8   mobile_wt      2000 non-null   int64
 9   n_cores        2000 non-null   int64
 10  pc             2000 non-null   int64
 11  px_height      2000 non-null   int64
 12  px_width       2000 non-null   int64
 13  ram            2000 non-null   int64
 14  sc_h           2000 non-null   int64
 15  sc_w           2000 non-null   int64
 16  talk_time      2000 non-null   int64
 17  three_g        2000 non-null   int64
 18  touch_screen   2000 non-null   int64
 19  wifi           2000 non-null   int64
 20  price_range    2000 non-null   int64
dtypes: float64(2), int64(19)
memory usage: 328.3 KB
```

2.1 Validaciones

Validando la existencia de valores nulos:

[6]:
```
count_null = data.isnull().sum()
count_null
```

[6]:
```
battery_power    0
blue             0
clock_speed      0
dual_sim         0
fc               0
four_g           0
int_memory       0
m_dep            0
mobile_wt        0
n_cores          0
pc               0
px_height        0
px_width         0
ram              0
sc_h             0
sc_w             0
talk_time        0
three_g          0
touch_screen     0
wifi             0
price_range      0
dtype: int64
```

3 1. Análisis Exploratorio de Datos (EDA)

3.1 Identificación de los diferentes tipos de variables

[7]:
```
# Variables numéricas discretas
int_vars = list(data.select_dtypes(include='int'))

# Variables categóricas (binarias -> dicotómicas)
categorical_variables = ['blue', 'dual_sim', 'four_g', 'three_g',␣
 ↪'touch_screen', 'wifi']
print(f'Variables categóricas (binarias ->␣
 ↪dicotómicas)\n{categorical_variables}')

# Variables numéricas discretas
discreet_vars = [var for var in int_vars if var not in categorical_variables]
discreet_vars.remove('price_range')  # Eliminamos la variable objetivo
print(f'Variables numéricas discretas\n{discreet_vars}')

# Variables numéricas continuas
continuous_vars = list(set(data.columns) - set(categorical_variables) -␣
 ↪set(discreet_vars) - {'price_range'})
print(f'Variables numéricas continuas\n{continuous_vars}')
```

```
Variables categóricas (binarias -> dicotómicas)
['blue', 'dual_sim', 'four_g', 'three_g', 'touch_screen', 'wifi']
Variables numéricas discretas
['battery_power', 'fc', 'int_memory', 'mobile_wt', 'n_cores', 'pc', 'px_height',
'px_width', 'ram', 'sc_h', 'sc_w', 'talk_time']
Variables numéricas continuas
['m_dep', 'clock_speed']
```

3.2 Variables numéricas

[8]:
```
#Resumen estadístico de las variables
data.describe().transpose()
```

[8]:

	count	mean	std	min	25%	50%	\
battery_power	2000.0	1238.51850	439.418206	501.0	851.75	1226.0	
blue	2000.0	0.49500	0.500100	0.0	0.00	0.0	
clock_speed	2000.0	1.52225	0.816004	0.5	0.70	1.5	
dual_sim	2000.0	0.50950	0.500035	0.0	0.00	1.0	
fc	2000.0	4.30950	4.341444	0.0	1.00	3.0	
four_g	2000.0	0.52150	0.499662	0.0	0.00	1.0	
int_memory	2000.0	32.04650	18.145715	2.0	16.00	32.0	
m_dep	2000.0	0.50175	0.288416	0.1	0.20	0.5	
mobile_wt	2000.0	140.24900	35.399655	80.0	109.00	141.0	
n_cores	2000.0	4.52050	2.287837	1.0	3.00	4.0	
pc	2000.0	9.91650	6.064315	0.0	5.00	10.0	

```
px_height      2000.0   645.10800   443.780811    0.0   282.75   564.0
px_width       2000.0  1251.51550   432.199447  500.0   874.75  1247.0
ram            2000.0  2124.21300  1084.732044  256.0  1207.50  2146.5
sc_h           2000.0    12.30650     4.213245    5.0     9.00    12.0
sc_w           2000.0     5.76700     4.356398    0.0     2.00     5.0
talk_time      2000.0    11.01100     5.463955    2.0     6.00    11.0
three_g        2000.0     0.76150     0.426273    0.0     1.00     1.0
touch_screen   2000.0     0.50300     0.500116    0.0     0.00     1.0
wifi           2000.0     0.50700     0.500076    0.0     0.00     1.0
price_range    2000.0     1.50000     1.118314    0.0     0.75     1.5

                   75%     max
battery_power  1615.25  1998.0
blue              1.00     1.0
clock_speed       2.20     3.0
dual_sim          1.00     1.0
fc                7.00    19.0
four_g            1.00     1.0
int_memory       48.00    64.0
m_dep             0.80     1.0
mobile_wt       170.00   200.0
n_cores           7.00     8.0
pc               15.00    20.0
px_height       947.25  1960.0
px_width       1633.00  1998.0
ram            3064.50  3998.0
sc_h             16.00    19.0
sc_w              9.00    18.0
talk_time        16.00    20.0
three_g           1.00     1.0
touch_screen      1.00     1.0
wifi              1.00     1.0
price_range       2.25     3.0
```

[9]:
```
#Resumen estadístico ampliado con mean, median y mode
desc = data.describe(include='all').transpose()

# media y mediana (solo para numéricas; el resto queda NaN)
desc['mean'] = data.mean(numeric_only=True)
desc['median'] = data.median(numeric_only=True)

# moda: tomar el primer valor de la(s) moda(s) por columna (funciona para␣
 ↪numéricas y categóricas)
modes = data.mode(dropna=True)
if not modes.empty:
    first_mode = modes.iloc[0]
else:
```

```
    first_mode = pd.Series(index=data.columns, dtype=object)
desc['mode'] = first_mode

# reordenar columnas para presentación (opcional)
cols_order =␣
 ↪['count','mean','median','mode','std','min','25%','50%','75%','max','unique','top','freq']
desc = desc.reindex(columns=[c for c in cols_order if c in desc.columns])

desc
```

[9]:

	count	mean	median	mode	std	min	\
battery_power	2000.0	1238.51850	1226.0	618.0	439.418206	501.0	
blue	2000.0	0.49500	0.0	0.0	0.500100	0.0	
clock_speed	2000.0	1.52225	1.5	0.5	0.816004	0.5	
dual_sim	2000.0	0.50950	1.0	1.0	0.500035	0.0	
fc	2000.0	4.30950	3.0	0.0	4.341444	0.0	
four_g	2000.0	0.52150	1.0	1.0	0.499662	0.0	
int_memory	2000.0	32.04650	32.0	27.0	18.145715	2.0	
m_dep	2000.0	0.50175	0.5	0.1	0.288416	0.1	
mobile_wt	2000.0	140.24900	141.0	182.0	35.399655	80.0	
n_cores	2000.0	4.52050	4.0	4.0	2.287837	1.0	
pc	2000.0	9.91650	10.0	10.0	6.064315	0.0	
px_height	2000.0	645.10800	564.0	347.0	443.780811	0.0	
px_width	2000.0	1251.51550	1247.0	874.0	432.199447	500.0	
ram	2000.0	2124.21300	2146.5	1229.0	1084.732044	256.0	
sc_h	2000.0	12.30650	12.0	17.0	4.213245	5.0	
sc_w	2000.0	5.76700	5.0	1.0	4.356398	0.0	
talk_time	2000.0	11.01100	11.0	7.0	5.463955	2.0	
three_g	2000.0	0.76150	1.0	1.0	0.426273	0.0	
touch_screen	2000.0	0.50300	1.0	1.0	0.500116	0.0	
wifi	2000.0	0.50700	1.0	1.0	0.500076	0.0	
price_range	2000.0	1.50000	1.5	0.0	1.118314	0.0	

	25%	50%	75%	max
battery_power	851.75	1226.0	1615.25	1998.0
blue	0.00	0.0	1.00	1.0
clock_speed	0.70	1.5	2.20	3.0
dual_sim	0.00	1.0	1.00	1.0
fc	1.00	3.0	7.00	19.0
four_g	0.00	1.0	1.00	1.0
int_memory	16.00	32.0	48.00	64.0
m_dep	0.20	0.5	0.80	1.0
mobile_wt	109.00	141.0	170.00	200.0
n_cores	3.00	4.0	7.00	8.0
pc	5.00	10.0	15.00	20.0
px_height	282.75	564.0	947.25	1960.0
px_width	874.75	1247.0	1633.00	1998.0

```
ram            1207.50  2146.5  3064.50  3998.0
sc_h              9.00    12.0    16.00    19.0
sc_w              2.00     5.0     9.00    18.0
talk_time         6.00    11.0    16.00    20.0
three_g           1.00     1.0     1.00     1.0
touch_screen      0.00     1.0     1.00     1.0
wifi              0.00     1.0     1.00     1.0
price_range       0.75     1.5     2.25     3.0
```

[10]:
```
data[discreet_vars]
```

[10]:
```
      battery_power  fc  int_memory  mobile_wt  n_cores  pc  px_height  \
0               842   1           7        188        2   2         20
1              1021   0          53        136        3   6        905
2               563   2          41        145        5   6       1263
3               615   0          10        131        6   9       1216
4              1821  13          44        141        2  14       1208
…               …  ..          …         …       …  ..         …
1995            794   0           2        106        6  14       1222
1996           1965   0          39        187        4   3        915
1997           1911   1          36        108        8   3        868
1998           1512   4          46        145        5   5        336
1999            510   5          45        168        6  16        483

      px_width   ram  sc_h  sc_w  talk_time
0          756  2549     9     7         19
1         1988  2631    17     3          7
2         1716  2603    11     2          9
3         1786  2769    16     8         11
4         1212  1411     8     2         15
…         …    …    …    …         …
1995      1890   668    13     4         19
1996      1965  2032    11    10         16
1997      1632  3057     9     1          5
1998       670   869    18    10         19
1999       754  3919    19     4          2

[2000 rows x 12 columns]
```

3.2.1 1.1. Escriba el código que permita crear la matriz de correlación de todas las variables numéricas:

[11]:
```
corr = data.corr()
corr
```

[11]:
```
                battery_power      blue  clock_speed  dual_sim        fc  \
battery_power        1.000000  0.011252     0.011482 -0.041847  0.033334
```

blue	0.011252	1.000000	0.021419	0.035198	0.003593
clock_speed	0.011482	0.021419	1.000000	-0.001315	-0.000434
dual_sim	-0.041847	0.035198	-0.001315	1.000000	-0.029123
fc	0.033334	0.003593	-0.000434	-0.029123	1.000000
four_g	0.015665	0.013443	-0.043073	0.003187	-0.016560
int_memory	-0.004004	0.041177	0.006545	-0.015679	-0.029133
m_dep	0.034085	0.004049	-0.014364	-0.022142	-0.001791
mobile_wt	0.001844	-0.008605	0.012350	-0.008979	0.023618
n_cores	-0.029727	0.036161	-0.005724	-0.024658	-0.013356
pc	0.031441	-0.009952	-0.005245	-0.017143	0.644595
px_height	0.014901	-0.006872	-0.014523	-0.020875	-0.009990
px_width	-0.008402	-0.041533	-0.009476	0.014291	-0.005176
ram	-0.000653	0.026351	0.003443	0.041072	0.015099
sc_h	-0.029959	-0.002952	-0.029078	-0.011949	-0.011014
sc_w	-0.021421	0.000613	-0.007378	-0.016666	-0.012373
talk_time	0.052510	0.013934	-0.011432	-0.039404	-0.006829
three_g	0.011522	-0.030236	-0.046433	-0.014008	0.001793
touch_screen	-0.010516	0.010061	0.019756	-0.017117	-0.014828
wifi	-0.008343	-0.021863	-0.024471	0.022740	0.020085
price_range	0.200723	0.020573	-0.006606	0.017444	0.021998

	four_g	int_memory	m_dep	mobile_wt	n_cores	… \
battery_power	0.015665	-0.004004	0.034085	0.001844	-0.029727	…
blue	0.013443	0.041177	0.004049	-0.008605	0.036161	…
clock_speed	-0.043073	0.006545	-0.014364	0.012350	-0.005724	…
dual_sim	0.003187	-0.015679	-0.022142	-0.008979	-0.024658	…
fc	-0.016560	-0.029133	-0.001791	0.023618	-0.013356	…
four_g	1.000000	0.008690	-0.001823	-0.016537	-0.029706	…
int_memory	0.008690	1.000000	0.006886	-0.034214	-0.028310	…
m_dep	-0.001823	0.006886	1.000000	0.021756	-0.003504	…
mobile_wt	-0.016537	-0.034214	0.021756	1.000000	-0.018989	…
n_cores	-0.029706	-0.028310	-0.003504	-0.018989	1.000000	…
pc	-0.005598	-0.033273	0.026282	0.018844	-0.001193	…
px_height	-0.019236	0.010441	0.025263	0.000939	-0.006872	…
px_width	0.007448	-0.008335	0.023566	0.000090	0.024480	…
ram	0.007313	0.032813	-0.009434	-0.002581	0.004868	…
sc_h	0.027166	0.037771	-0.025348	-0.033855	-0.000315	…
sc_w	0.037005	0.011731	-0.018388	-0.020761	0.025826	…
talk_time	-0.046628	-0.002790	0.017003	0.006209	0.013148	…
three_g	0.584246	-0.009366	-0.012065	0.001551	-0.014733	…
touch_screen	0.016758	-0.026999	-0.002638	-0.014368	0.023774	…
wifi	-0.017620	0.006993	-0.028353	-0.000409	-0.009964	…
price_range	0.014772	0.044435	0.000853	-0.030302	0.004399	…

	px_height	px_width	ram	sc_h	sc_w	talk_time \
battery_power	0.014901	-0.008402	-0.000653	-0.029959	-0.021421	0.052510
blue	-0.006872	-0.041533	0.026351	-0.002952	0.000613	0.013934

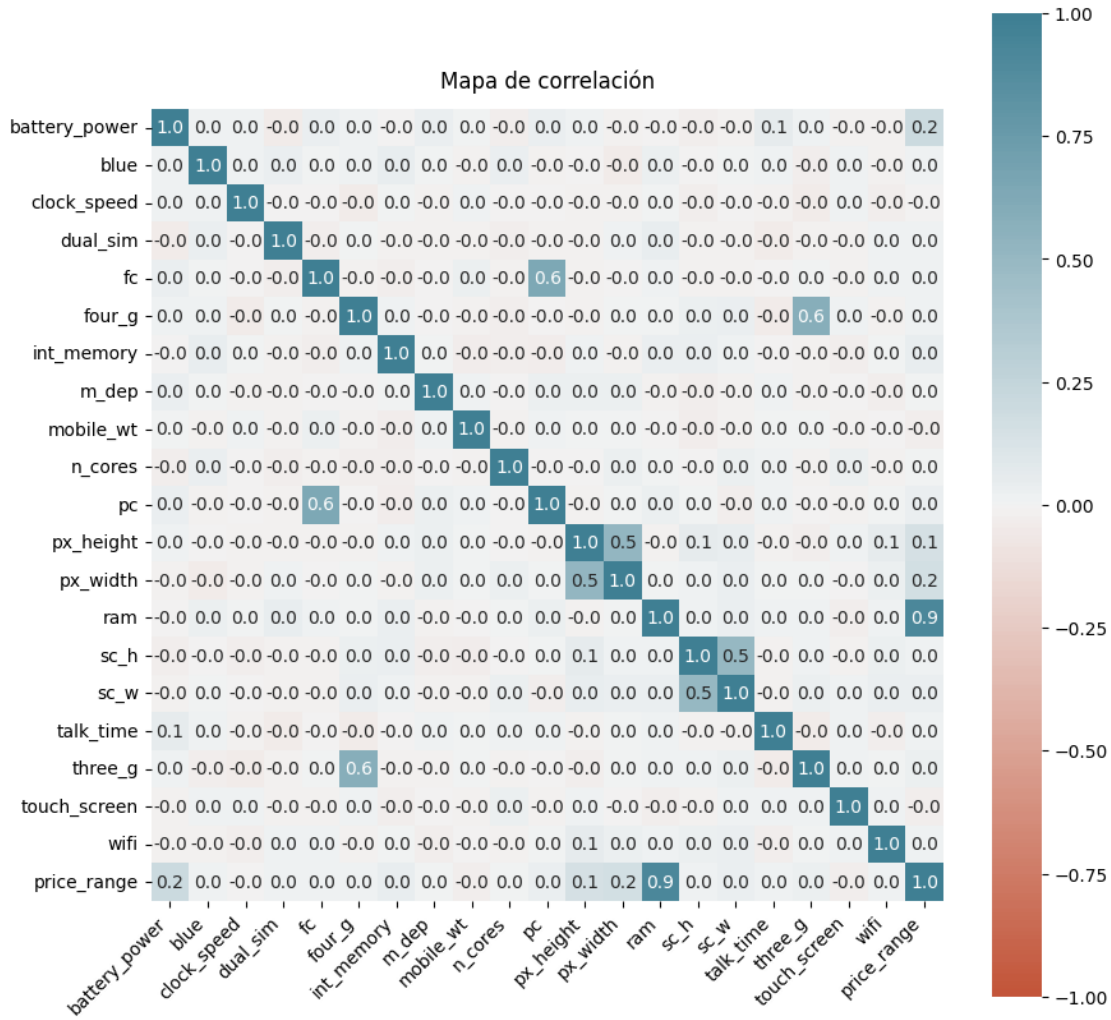
clock_speed	-0.014523	-0.009476	0.003443	-0.029078	-0.007378	-0.011432
dual_sim	-0.020875	0.014291	0.041072	-0.011949	-0.016666	-0.039404
fc	-0.009990	-0.005176	0.015099	-0.011014	-0.012373	-0.006829
four_g	-0.019236	0.007448	0.007313	0.027166	0.037005	-0.046628
int_memory	0.010441	-0.008335	0.032813	0.037771	0.011731	-0.002790
m_dep	0.025263	0.023566	-0.009434	-0.025348	-0.018388	0.017003
mobile_wt	0.000939	0.000090	-0.002581	-0.033855	-0.020761	0.006209
n_cores	-0.006872	0.024480	0.004868	-0.000315	0.025826	0.013148
pc	-0.018465	0.004196	0.028984	0.004938	-0.023819	0.014657
px_height	1.000000	0.510664	-0.020352	0.059615	0.043038	-0.010645
px_width	0.510664	1.000000	0.004105	0.021599	0.034699	0.006720
ram	-0.020352	0.004105	1.000000	0.015996	0.035576	0.010820
sc_h	0.059615	0.021599	0.015996	1.000000	0.506144	-0.017335
sc_w	0.043038	0.034699	0.035576	0.506144	1.000000	-0.022821
talk_time	-0.010645	0.006720	0.010820	-0.017335	-0.022821	1.000000
three_g	-0.031174	0.000350	0.015795	0.012033	0.030941	-0.042688
touch_screen	0.021891	-0.001628	-0.030455	-0.020023	0.012720	0.017196
wifi	0.051824	0.030319	0.022669	0.025929	0.035423	-0.029504
price_range	0.148858	0.165818	0.917046	0.022986	0.038711	0.021859

	three_g	touch_screen	wifi	price_range
battery_power	0.011522	-0.010516	-0.008343	0.200723
blue	-0.030236	0.010061	-0.021863	0.020573
clock_speed	-0.046433	0.019756	-0.024471	-0.006606
dual_sim	-0.014008	-0.017117	0.022740	0.017444
fc	0.001793	-0.014828	0.020085	0.021998
four_g	0.584246	0.016758	-0.017620	0.014772
int_memory	-0.009366	-0.026999	0.006993	0.044435
m_dep	-0.012065	-0.002638	-0.028353	0.000853
mobile_wt	0.001551	-0.014368	-0.000409	-0.030302
n_cores	-0.014733	0.023774	-0.009964	0.004399
pc	-0.001322	-0.008742	0.005389	0.033599
px_height	-0.031174	0.021891	0.051824	0.148858
px_width	0.000350	-0.001628	0.030319	0.165818
ram	0.015795	-0.030455	0.022669	0.917046
sc_h	0.012033	-0.020023	0.025929	0.022986
sc_w	0.030941	0.012720	0.035423	0.038711
talk_time	-0.042688	0.017196	-0.029504	0.021859
three_g	1.000000	0.013917	0.004316	0.023611
touch_screen	0.013917	1.000000	0.011917	-0.030411
wifi	0.004316	0.011917	1.000000	0.018785
price_range	0.023611	-0.030411	0.018785	1.000000

[21 rows x 21 columns]

3.2.2 Graficando la Correlación

[12]:
```
plt.figure(figsize=(10, 10))
ax = sns.heatmap(
    corr,
    vmin=-1, vmax=1, center=0,
    cmap=sns.diverging_palette(20, 220, n=200),
    square=True,
    annot = True, fmt=".1f"
)
ax.set_xticklabels(
    ax.get_xticklabels(),
    rotation=45,
    horizontalalignment='right'
);

ax.set_title('Mapa de correlación', fontdict={'fontsize':12}, pad=12);
```



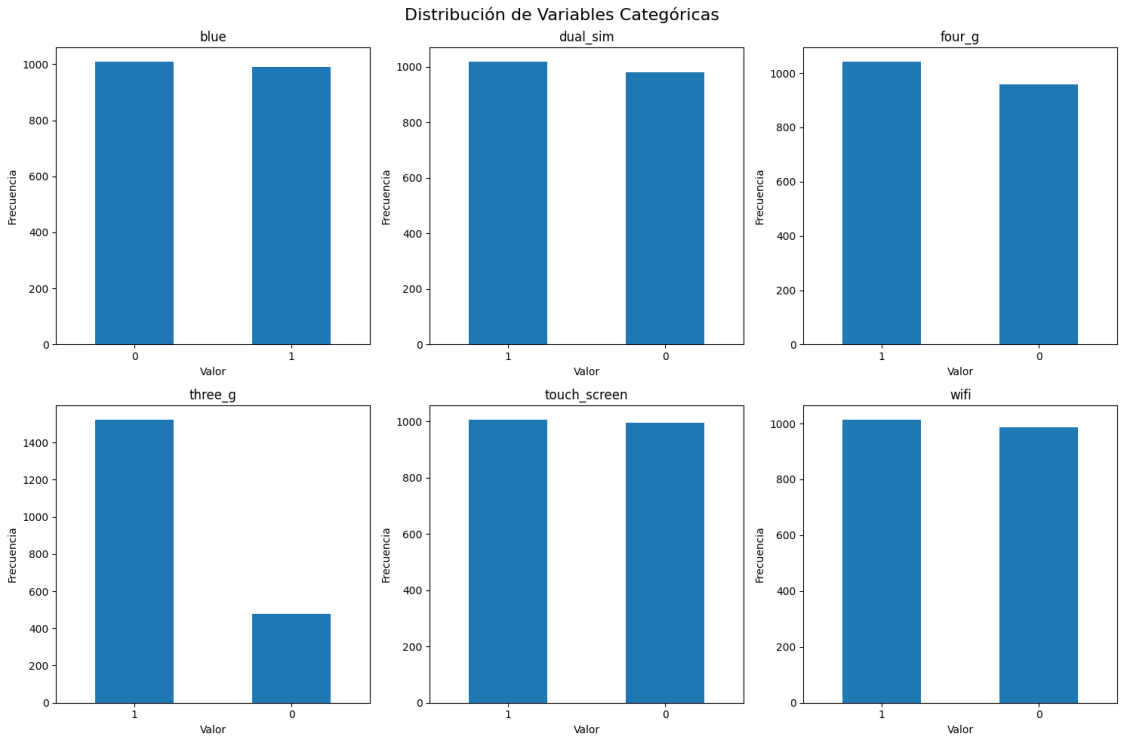
3.3 Variables catégoricas

3.3.1 1.2. Escriba el código para su análisis y la creación de gráficos de frecuencia:

[13]:
```
# Replaced plotting block with a dynamic-layout version
import math

# Análisis de frecuencias de cada variable categórica
for variable in categorical_variables:
    print(f"\nFrecuencia de {variable}:")
    print(data[variable].value_counts())

# Gráfico de frecuencia para cada variable categórica
n = len(categorical_variables)
ncols = 3
nrows = math.ceil(n / ncols)

fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(15, 5 * nrows))
fig.suptitle('Distribución de Variables Categóricas', fontsize=16)

# axes can be 2D or 1D; flatten for uniform handling
axes_flat = axes.flatten() if hasattr(axes, "flatten") else np.array([axes])

for idx, variable in enumerate(categorical_variables):
    ax = axes_flat[idx]
    counts = data[variable].value_counts()
    counts.plot(kind='bar', ax=ax)
    ax.set_title(variable)
    ax.set_xlabel('Valor')
    ax.set_ylabel('Frecuencia')
    labels = [str(x) for x in counts.index]
    ax.set_xticklabels(labels, rotation=0)

# hide any unused subplots
for ax in axes_flat[n:]:
    ax.axis('off')

plt.tight_layout()
plt.show()
```

```
Frecuencia de blue:
blue
0    1010
1     990
Name: count, dtype: int64
```

```
Frecuencia de dual_sim:
dual_sim
1    1019
0     981
Name: count, dtype: int64

Frecuencia de four_g:
four_g
1    1043
0     957
Name: count, dtype: int64

Frecuencia de three_g:
three_g
1    1523
0     477
Name: count, dtype: int64

Frecuencia de touch_screen:
touch_screen
1    1006
0     994
Name: count, dtype: int64

Frecuencia de wifi:
wifi
1    1014
0     986
Name: count, dtype: int64
```

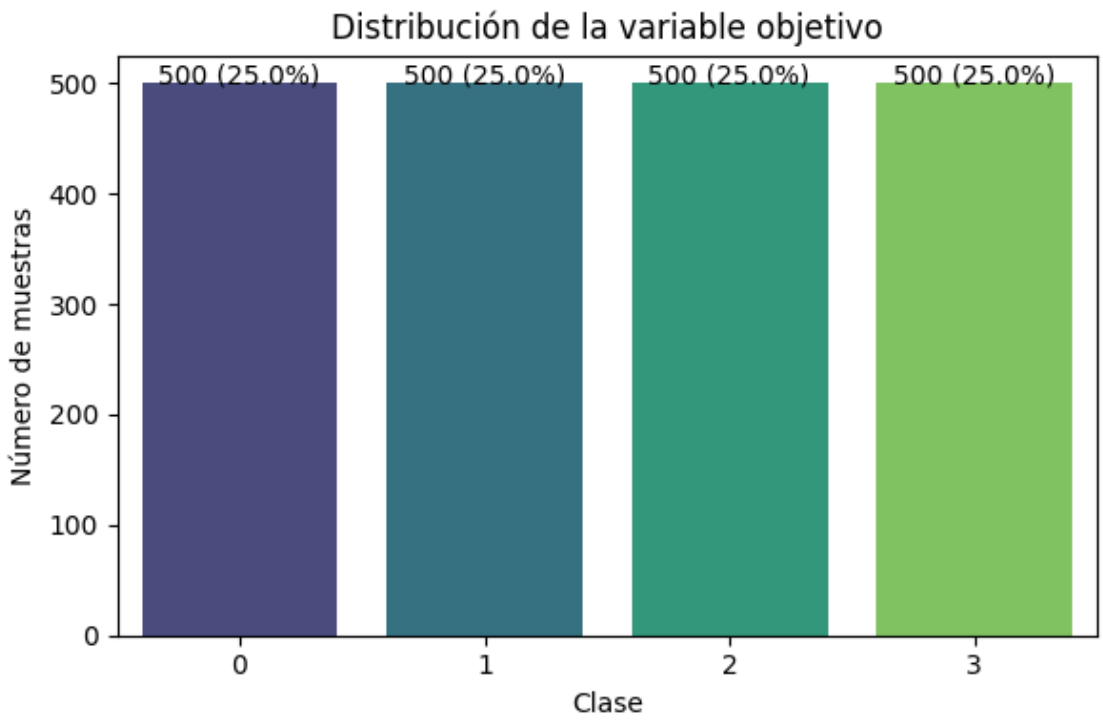


3.3.2 1.3. Escriba el código que permita validar si el dataset está balanceado

[14]:
```
# sustituir 'price_range' por la columna objetivo
variable_objetivo = data['price_range']

# conteos y porcentajes
counts = variable_objetivo.value_counts().sort_index()
pct = (counts / counts.sum()) * 100
df_dist = pd.DataFrame({'count': counts, 'percent': pct})
print(df_dist)

# visualización
x_values = counts.index.astype(str)
y_values = counts.values

plt.figure(figsize=(6,4))
sns.barplot(x=x_values, y=y_values, palette='viridis', legend=False,␣
 ↪hue=x_values)
plt.title('Distribución de la variable objetivo')
plt.xlabel('Clase')
plt.ylabel('Número de muestras')
for i,v in enumerate(counts.values):
    plt.text(i, v+0.5, f"{v} ({pct.iloc[i]:.1f}%)", ha='center')
```

```
plt.tight_layout()
plt.show()
```

```
             count  percent
price_range
0              500     25.0
1              500     25.0
2              500     25.0
3              500     25.0
```



3.4 PREPROCESAMIENTO DE DATOS

[15]:
```
#Crear una copia del dataset para modificaciones
data2 = data.copy()
```

[16]:
```
#Verificar tamaño
data2.shape
```

[16]: (2000, 21)

3.4.1 División de datos

[17]:
```
# Separación de los datos en train y test
X = data2.drop(columns = 'price_range')
y = data2['price_range']

X_train, X_test, y_train, y_test = train_test_split(
                                        X,
                                        y,
                                        train_size   = 0.8,
                                        random_state = seed,
                                        shuffle      = True
                                    )
```

3.4.2 Escriba el código que permita reescalar todos los atributos del dataset utilizando la función del StandardScaler:

[18]:
```
# Código para reescalar X_train y X_test mediante StandardScaler
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

4 2. Aplicación de SVM sobre los datos

Importante:

A partir de esta sección, utiliza los datos estandarizados.

4.0.1 2.1. Escriba el código que permita crear un modelo utilizando SVM con kernel lineal, el valor de C =1 y random_state=seed

[19]:
```
# Creación del modelo SVM lineal
modeloSVM = SVC(C=1, kernel='linear', random_state=seed)
modeloSVM.fit(X_train, y_train)
```

[19]:
```
SVC(C=1, kernel='linear', random_state=1234)
```

[20]:
```
# Predicciones test
predictSVM = modeloSVM.predict(X_test)
predictSVM
```

[20]:
```
array([1, 2, 2, 2, 2, 2, 1, 3, 1, 3, 3, 3, 0, 2, 3, 0, 3, 2, 2, 2, 0, 1,
       1, 0, 3, 2, 0, 3, 1, 0, 2, 0, 3, 3, 3, 3, 3, 3, 0, 2, 3, 1, 3, 2,
       1, 0, 2, 2, 0, 3, 3, 0, 1, 2, 1, 3, 3, 1, 2, 1, 1, 2, 2, 1, 2, 1,
       0, 3, 3, 3, 0, 3, 1, 3, 1, 2, 1, 2, 2, 3, 3, 2, 1, 0, 2, 3, 2, 3,
       3, 1, 1, 2, 0, 1, 3, 0, 1, 2, 2, 2, 0, 1, 0, 2, 3, 2, 3, 3, 1, 1,
```

```
       2, 2, 0, 1, 2, 3, 3, 2, 2, 1, 1, 0, 1, 2, 3, 1, 2, 1, 1, 0, 1, 1,
       0, 0, 0, 1, 3, 1, 2, 0, 3, 3, 1, 1, 3, 0, 2, 0, 2, 0, 0, 3, 0, 3,
       1, 3, 2, 2, 1, 2, 2, 3, 0, 2, 2, 1, 1, 2, 0, 0, 1, 3, 1, 1, 3, 1,
       3, 0, 2, 2, 3, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 2, 2, 0, 1, 0,
       1, 1, 2, 3, 3, 0, 2, 1, 3, 3, 1, 3, 1, 0, 3, 1, 2, 3, 3, 2, 3, 3,
       3, 2, 0, 3, 0, 3, 1, 0, 2, 3, 0, 0, 2, 1, 3, 1, 3, 1, 1, 1, 3, 1,
       2, 0, 2, 2, 3, 2, 3, 1, 1, 3, 1, 3, 2, 0, 2, 1, 0, 2, 3, 1, 0, 0,
       0, 0, 1, 3, 2, 2, 1, 0, 3, 0, 0, 2, 3, 0, 3, 2, 1, 1, 3, 0, 0, 2,
       0, 0, 0, 1, 0, 2, 2, 1, 2, 0, 0, 0, 0, 3, 3, 2, 0, 0, 3, 1, 3, 2,
       0, 0, 1, 2, 3, 3, 2, 3, 3, 2, 1, 1, 3, 2, 1, 3, 3, 2, 2, 0, 3, 1,
       0, 2, 1, 3, 0, 0, 2, 1, 1, 0, 2, 3, 3, 1, 3, 1, 0, 3, 3, 1, 2, 1,
       3, 1, 0, 0, 3, 1, 0, 2, 3, 2, 3, 0, 1, 2, 3, 2, 3, 2, 2, 1, 0, 1,
       3, 2, 3, 1, 1, 2, 3, 0, 0, 3, 0, 1, 3, 0, 1, 1, 3, 0, 2, 3, 3, 0,
       1, 0, 3, 0])
```

4.0.2 2.2. Escriba el código que permita hallar la exactitud (accuracy) del modelo SVM anteriormente entrenado:

[21]:
```python
accuracy_svm = accuracy_score(y_test, predictSVM)
print(f'Exactitud del modelo SVM: {accuracy_svm:.4f}')
```

```
Exactitud del modelo SVM: 0.9675
```

[22]:
```python
class_labels = ['Bajo', 'Medio', 'Alto', 'Muy Alto']
print(classification_report(y_test, predictSVM, target_names=class_labels))
```

```
              precision    recall  f1-score   support

        Bajo       0.98      0.99      0.98        93
       Medio       0.96      0.95      0.96       102
        Alto       0.96      0.94      0.95        98
    Muy Alto       0.97      0.99      0.98       107

    accuracy                           0.97       400
   macro avg       0.97      0.97      0.97       400
weighted avg       0.97      0.97      0.97       400
```

4.0.3 2.3. Escriba el código que permita encontrar los mejores parámetros para el modelo SVM, con un param_grid = {'C': np.linspace(0.1, 100, 20), 'kernel': ('linear', 'rbf')

Ajuste de hiperparámetros (GridSearchCV) Restricción:

No imprimas los resultados del GridSearchCV. Solo utiliza los mejores parámetros encontrados.

[23]:
```
# Para que no se impriman los resultados, estos se asignan a _
from sklearn.model_selection import GridSearchCV

# Parametros y los valores que van a tomar
param_grid = {'C': np.linspace(0.1, 100, 20),
              'kernel': ['linear', 'rbf']
             }

grid = GridSearchCV(SVC(), param_grid, refit = True, verbose = 0)

# fitting the model for grid search
# Para que no se impriman los resultados, estos se asignan a _
_ = grid.fit(X_train, y_train)

# Se asignan los resultados a un dataframe
resultados = pd.DataFrame(grid.cv_results_)
resultados.filter(regex = '(param.*|mean_t|std_t)')\
    .drop(columns = 'params')\
    .sort_values('mean_test_score', ascending = False) \
    .head(5)
```

[23]:

	param_C	param_kernel	mean_test_score	std_test_score
16	42.163158	linear	0.969375	0.011075
18	47.421053	linear	0.969375	0.012087
14	36.905263	linear	0.969375	0.008705
10	26.389474	linear	0.968750	0.010643
22	57.936842	linear	0.968750	0.010643

4.0.4 2.4. Escriba el código que permita conocer los mejores parámetros encontrados en el ítem anterior

[24]:
```
# print best parameter after tuning
print(f'Mejores hiperparámetros {grid.best_params_}')
```

```
Mejores hiperparámetros {'C': np.float64(36.905263157894744), 'kernel':
'linear'}
```

4.0.5 2.5. Escriba el código que permita hallar la exactitud del modelo de SVM aplicando los mejores parámetros

[25]:
```
# Accuracy de test del modelo
# ==============================================================================

best_predictor = grid.best_estimator_
print(best_predictor.get_params())
best_prediction = best_predictor.predict(X_test)
accuracy_best = accuracy_score(y_test, best_prediction)
```

```
print(f'Exactitud del mejor modelo tras ajuste de hiperparámetros:␣
↪{accuracy_best:.4f}')
```

```
{'C': np.float64(36.905263157894744), 'break_ties': False, 'cache_size': 200,
'class_weight': None, 'coef0': 0.0, 'decision_function_shape': 'ovr', 'degree':
3, 'gamma': 'scale', 'kernel': 'linear', 'max_iter': -1, 'probability': False,
'random_state': None, 'shrinking': True, 'tol': 0.001, 'verbose': False}
Exactitud del mejor modelo tras ajuste de hiperparámetros: 0.9650
```

4.0.6 2.6. Escriba el código que permita hallar la las métricas del modelo entrenado en el ítem anterior. Utilice la función classification_report

[26]:
```
from sklearn.metrics import classification_report
class_labels = ['Bajo', 'Medio', 'Alto', 'Muy Alto']
print(classification_report(y_test, predictSVM, target_names=class_labels))
```

```
              precision    recall  f1-score   support

        Bajo       0.98      0.99      0.98        93
       Medio       0.96      0.95      0.96       102
        Alto       0.96      0.94      0.95        98
    Muy Alto       0.97      0.99      0.98       107

    accuracy                           0.97       400
   macro avg       0.97      0.97      0.97       400
weighted avg       0.97      0.97      0.97       400
```

5 3. REDES NEURONALES

Recordatorio:

- Mantén los datos normalizados (los mismos que usaste en SVM).
- No regreses al dataset original ni repitas pasos anteriores, a menos que se indique explícitamente.

5.0.1 3.1. Escriba el código que permita entrenar un perceptrón multicapa con 3 capas ocultas con 200, 100 y 50 neuronas respectivamente en cada capa. Función de activación 'Relu' y random_state=seed

[27]:
```
from sklearn.neural_network import MLPClassifier

classifier = MLPClassifier(hidden_layer_sizes=(200, 100, 50),␣
↪activation='relu', random_state=seed)
classifier.fit(X_train, y_train)

predictMLP = classifier.predict(X_test)
```

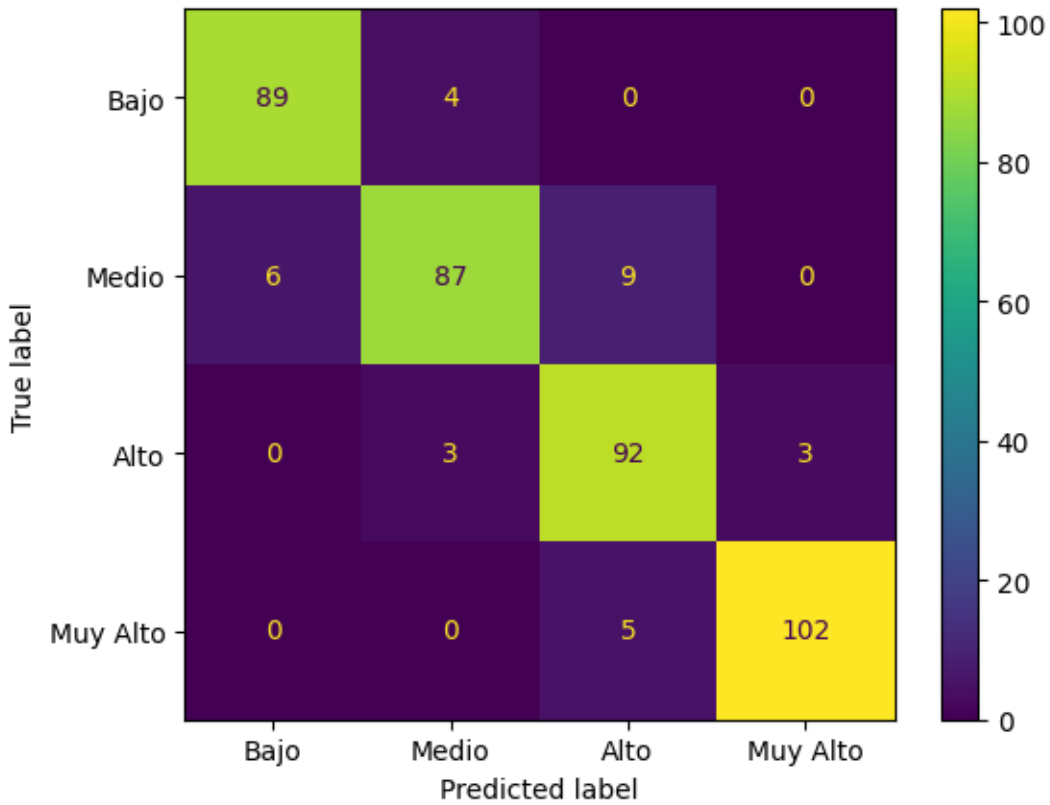
```
predictMLP
```

[27]:
```
array([1, 2, 2, 2, 2, 2, 2, 3, 1, 3, 3, 3, 0, 2, 3, 0, 3, 2, 2, 2, 0, 1,
       1, 0, 3, 2, 0, 3, 1, 0, 2, 0, 3, 3, 3, 3, 3, 3, 0, 2, 3, 1, 3, 3,
       1, 0, 2, 2, 0, 3, 3, 0, 1, 2, 1, 3, 3, 1, 2, 1, 1, 2, 2, 1, 2, 1,
       0, 3, 3, 3, 0, 3, 1, 3, 1, 2, 1, 2, 2, 3, 3, 2, 1, 0, 2, 3, 2, 3,
       3, 2, 1, 2, 0, 1, 3, 0, 1, 2, 2, 2, 0, 1, 0, 2, 3, 2, 3, 2, 1, 1,
       1, 2, 0, 1, 2, 3, 3, 2, 2, 1, 1, 0, 1, 2, 3, 1, 2, 1, 1, 0, 1, 1,
       0, 0, 0, 1, 3, 2, 2, 0, 3, 3, 1, 1, 3, 0, 2, 1, 2, 0, 0, 3, 0, 3,
       1, 3, 2, 2, 0, 2, 2, 3, 0, 2, 2, 1, 2, 2, 0, 0, 1, 3, 2, 1, 3, 1,
       3, 0, 2, 2, 3, 1, 1, 1, 0, 0, 2, 2, 0, 1, 0, 0, 0, 2, 2, 0, 1, 0,
       1, 1, 1, 2, 3, 0, 2, 1, 3, 3, 1, 3, 0, 0, 3, 1, 1, 3, 3, 2, 3, 3,
       3, 2, 0, 3, 0, 3, 1, 0, 2, 3, 0, 0, 2, 1, 3, 1, 3, 1, 2, 1, 2, 2,
       2, 0, 2, 2, 3, 2, 3, 0, 1, 3, 1, 3, 2, 0, 2, 2, 0, 1, 3, 1, 0, 0,
       0, 0, 1, 3, 2, 2, 1, 0, 3, 0, 0, 2, 3, 0, 3, 2, 1, 1, 3, 0, 0, 2,
       0, 0, 0, 1, 0, 2, 2, 1, 2, 0, 0, 0, 0, 3, 3, 2, 0, 0, 3, 1, 3, 2,
       0, 0, 1, 2, 2, 3, 2, 3, 3, 2, 1, 1, 2, 2, 1, 3, 3, 2, 2, 0, 3, 1,
       0, 2, 1, 3, 0, 0, 2, 1, 1, 0, 2, 3, 3, 1, 3, 1, 0, 3, 3, 1, 2, 1,
       3, 1, 0, 0, 3, 1, 0, 2, 3, 2, 3, 0, 1, 2, 3, 2, 3, 2, 2, 0, 0, 1,
       3, 2, 3, 1, 1, 2, 3, 0, 0, 3, 0, 1, 3, 0, 1, 1, 3, 0, 2, 3, 3, 1,
       1, 0, 3, 0])
```

5.0.2 3.2. Escriba el código que permita hallar la exactitud del modelo de redes neuronales anteriormente entrenado:

[28]:
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

accuracy_mlp = accuracy_score(y_test, predictMLP)
print(f'Exactitud del modelo MLP: {accuracy_mlp:.4f}')

value_labels = [0, 1, 2, 3]

cm = confusion_matrix(y_test, predictMLP, labels=value_labels)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_labels)
disp.plot();
```

```
Exactitud del modelo MLP: 0.9250
```



5.0.3 3.3. Escriba el código que permita hallar la las métricas del modelo entrenado en el ítem anterior. Utilice la función classification_report

[29]:
```
from sklearn.metrics import classification_report
print(classification_report(y_test, predictMLP, target_names=class_labels))
```

```
              precision    recall  f1-score   support

        Bajo       0.94      0.96      0.95        93
       Medio       0.93      0.85      0.89       102
        Alto       0.87      0.94      0.90        98
    Muy Alto       0.97      0.95      0.96       107

    accuracy                           0.93       400
   macro avg       0.93      0.93      0.92       400
weighted avg       0.93      0.93      0.92       400
```

5.0.4 4. Conclusiones

Escribe tus conclusiones finales, reflexionando sobre los resultados obtenidos con SVM y MLP. Compara los siguientes aspectos:

1. Métricas relevantes:

- ¿Cuáles métricas fueron clave para evaluar el desempeño?

2. Fortalezas y limitaciones:

- Ventajas y desventajas de cada modelo en este caso específico.

5.0.5 Métricas relevantes

- Para el modelo SVM se utilizó el ajuste de hiperpárametros (GridSearchVC) para la evaluación del usando hiperparámetro
- Para el modelo MLP se utilizó la matriz de confusión para la evaluación de falsos positivos / negativos
- Para ambos modelos se utilizaron las funciones `accuracy_score` y `classification_report`

5.0.6 Fortalezas y limitaciones

- Para ambos casos los modelos utilizados fueron para la evaluación de regresiones
- Para este caso los datos se encontraban balanceados
- En comparación el modelo SVM presentaba una mayor exactitud en contraste con el MLP

5.0.7 Comentarios

- Se observo una alta correlación directa entre la memoria RAM y el rango de precio

[ ]:

```
```